

UK Climate Insights API — Documentation

Version: 1.0.0 **Base URL:** `http://localhost:8000` (local) / `https://<deployed-host>` (production) **Format:** JSON (all requests and responses)

Interactive docs: `/docs` (Swagger UI) · `/redoc` (ReDoc)

Table of Contents

- [1. Overview](#)
- [2. Authentication](#)
- [3. Pagination](#)
- [4. Error Codes](#)
- [5. Data Models](#)
- [6. Endpoints — Stations](#)
- [7. Endpoints — Observations](#)
- [8. Endpoints — Annotations](#)
- [9. Endpoints — Analytics](#)
- [10. Endpoints — System](#)
- [11. MCP Server](#)

Overview

The UK Climate Insights API provides access to historical UK weather observations from 8 Met Office monitoring stations, spanning from 1853 to 2026. It supports full CRUD operations on stations, observations, and annotations, plus a suite of analytical endpoints for trend analysis, anomaly detection, seasonal statistics, and climate normals.

Data sources:

- Met Office Historic Station Data (Open Government Licence v3.0)
- Stations: Armagh, Cambridge, Cardiff, Durham, Heathrow, Lerwick, Oxford, Sheffield

Tech stack: Python 3.13 · FastAPI 0.115 · PostgreSQL 16 · SQLAlchemy 2 (async) · Pydantic v2

Authentication

Read operations (`GET`) are **public** — no authentication required.

Write operations (`POST`, `PUT`, `PATCH`, `DELETE`) require an API key passed in the request header:

```
X-API-Key: your-api-key
```

Example:

```
curl -X POST http://localhost:8000/stations \
  -H "Content-Type: application/json" \
  -H "X-API-Key: change-me-in-production" \
  -d '{"station_id": "YORK", "name": "York", ...}'
```

Invalid or missing key response:

```
HTTP 401 Unauthorized

{
  "detail": "Invalid or missing API key"
}
```

To set your API key, configure `ADMIN_API_KEY` in your `.env` file.

Pagination

All list endpoints return paginated responses with the following structure:

```
{
  "data": [ ... ],
  "pagination": {
    "page": 1,
    "page_size": 50,
    "total": 11097,
    "pages": 222
  }
}
```

Query parameters (all list endpoints):

Parameter	Type	Default	Description
page	integer	1	Page number (1-indexed)
page_size	integer	50	Records per page (max 500)

Error Codes

Code	Meaning	When returned
200	OK	Successful GET/PATCH/PUT
201	Created	Successful POST
204	No Content	Successful DELETE
400	Bad Request	Malformed request body
401	Unauthorized	Missing or invalid API key
404	Not Found	Resource does not exist
409	Conflict	Duplicate resource (e.g. station_id already exists)
422	Unprocessable Entity	Validation error (wrong types, out-of-range values)
429	Too Many Requests	Rate limit exceeded on analytics endpoints
500	Internal Server Error	Unexpected server error

All error responses follow the format:

```
{
  "detail": "Human-readable error message"
}
```

Validation errors (422) return detailed field-level messages:

```
{
  "detail": [
    {
      "loc": ["body", "latitude"],
      "msg": "value is not a valid float",
      "type": "type_error.float"
    }
  ]
}
```

Data Models

Station

Field	Type	Description
id	integer	Internal database ID
station_id	string	Met Office station code (e.g. <code>HEATHROW</code>)
name	string	Human-readable station name
latitude	float	Decimal degrees (WGS84)
longitude	float	Decimal degrees (WGS84)
elevation_m	integer	Elevation above sea level (metres)
region	string	UK region (e.g. <code>Greater London</code>)
country	string	Country within UK (e.g. <code>England</code>)
opened_year	integer null	Year station opened
closed_year	integer null	Year station closed (null = still active)
created_at	datetime	Record creation timestamp
updated_at	datetime	Record last-updated timestamp

Observation

Field	Type	Description
id	integer	Internal database ID
station_id	integer	Foreign key to Station.id
date	date	Observation date (YYYY-MM-DD, always 15th of month)
max_temp_c	float null	Monthly mean daily maximum temperature (°C)
min_temp_c	float null	Monthly mean daily minimum temperature (°C)
mean_temp_c	float null	Monthly mean temperature (°C)
rainfall_mm	float null	Total monthly rainfall (mm)
sunshine_hours	float null	Total monthly sunshine hours
snow_depth_cm	float null	Mean snow depth (cm)
data_quality	integer	Quality flag: 1=good, 2=estimated, 3=suspect
created_at	datetime	Record creation timestamp
updated_at	datetime	Record last-updated timestamp

Annotation

Field	Type	Description
id	integer	Internal database ID
station_id	integer	Foreign key to Station.id
observation_date	date	Date the annotation refers to
author	string	Annotation author name
note	string	Annotation text (max 2000 characters)
approved	boolean	Whether annotation has been approved
created_at	datetime	Record creation timestamp
updated_at	datetime	Record last-updated timestamp

Stations

GET /stations — List all stations

Returns a paginated list of UK weather stations.

Query parameters:

Parameter	Type	Default	Description
region	string	—	Filter by region (partial match, case-insensitive)
country	string	—	Filter by country (partial match)
active_only	boolean	false	Only return stations still operating
page	integer	1	Page number
page_size	integer	50	Records per page

Example request:

```
GET /stations?country=Scotland&page=1&page_size=10
```

Example response:

```
{
  "data": [
    {
      "id": 1,
      "station_id": "ARMAGH",
      "name": "Armagh",
      "latitude": 54.352,
      "longitude": -6.649,
      "elevation_m": 62,
      "region": "Northern Ireland",
      "country": "Northern Ireland",
      "opened_year": 1853,
      "closed_year": null,
      "created_at": "2025-01-01T00:00:00",
      "updated_at": "2025-01-01T00:00:00"
    }
  ],
  "pagination": {
    "page": 1,
    "page_size": 10,
    "total": 1,
    "pages": 1
  }
}
```

GET /stations/

Path parameters:

Parameter	Type	Description
id	integer	Internal station ID

Example request:

GET /stations/5

Example response:

```
{
  "id": 5,
  "station_id": "HEATHROW",
  "name": "Heathrow",
  "latitude": 51.479,
  "longitude": -0.449,
  "elevation_m": 25,
  "region": "Greater London",
  "country": "England",
  "opened_year": 1948,
  "closed_year": null,
  "created_at": "2025-01-01T00:00:00",
  "updated_at": "2025-01-01T00:00:00"
}
```

Error responses: 404 if station not found.

POST /stations — Create a station (*requires API key*)

Request body:

Field	Type	Required	Constraints
station_id	string	yes	Unique, max 20 chars, uppercase
name	string	yes	Max 100 chars
latitude	float	yes	-90 to 90
longitude	float	yes	-180 to 180
elevation_m	integer	no	0 to 9999
region	string	no	Max 100 chars
country	string	no	Max 50 chars
opened_year	integer	no	1600 to 2100
closed_year	integer	no	1600 to 2100

Example request:

```
POST /stations
X-API-Key: change-me-in-production
Content-Type: application/json

{
  "station_id": "YORK",
  "name": "York",
  "latitude": 53.958,
  "longitude": -1.087,
  "elevation_m": 17,
  "region": "Yorkshire",
  "country": "England",
  "opened_year": 1900
}
```

Example response: 201 Created — returns the created Station object.

Error responses: 401 (bad key), 409 (station_id already exists), 422 (validation).

PUT /stations/

Full replacement of all fields. Same request body as POST (all required fields must be supplied).

Example request:

```
PUT /stations/9
X-API-Key: change-me-in-production
Content-Type: application/json

{
  "station_id": "YORK",
  "name": "York (Updated)",
  "latitude": 53.958,
  "longitude": -1.087
}
```

Error responses: 401, 404, 422.

PATCH /stations/

Supply only the fields to change.

Example request:

```
PATCH /stations/9
X-API-Key: change-me-in-production
Content-Type: application/json

{
  "closed_year": 2020
}
```

Error responses: 401, 404, 422.

DELETE /stations/

Deletes the station and all associated observations and annotations (cascade).

Example request:

```
DELETE /stations/9
X-API-Key: change-me-in-production
```

Response: 204 No Content (empty body).

Error responses: 401, 404.

Observations

GET /observations — List observations

Returns paginated monthly climate observations.

Query parameters:

Parameter	Type	Default	Description
station_id	integer	—	Filter by internal station ID
date_from	date	—	Start date (YYYY-MM-DD)
date_to	date	—	End date (YYYY-MM-DD)
data_quality	integer	—	1=good, 2=estimated, 3=suspect
page	integer	1	Page number
page_size	integer	50	Records per page

Example request:

```
GET /observations?station_id=5&date_from=2020-01-01&date_to=2020-12-31
```

Example response:

```
{
  "data": [
    {
      "id": 8834,
      "station_id": 5,
      "date": "2020-01-15",
      "max_temp_c": 9.4,
      "min_temp_c": 4.1,
      "mean_temp_c": 6.7,
      "rainfall_mm": 55.2,
      "sunshine_hours": 52.3,
      "snow_depth_cm": null,
      "data_quality": 1,
      "created_at": "2025-01-01T00:00:00",
      "updated_at": "2025-01-01T00:00:00"
    }
  ],
  "pagination": {
    "page": 1,
    "page_size": 50,
    "total": 12,
    "pages": 1
  }
}
```

GET /observations/

Example request:

```
GET /observations/8834
```

Returns the full Observation object. **Error responses:** 404.

POST /observations — Create an observation (*requires API key*)

Request body:

Field	Type	Required	Constraints
station_id	integer	yes	Must reference an existing station
date	date	yes	YYYY-MM-DD

Field	Type	Required	Constraints
max_temp_c	float	no	-70 to 60
min_temp_c	float	no	-70 to 60
mean_temp_c	float	no	-70 to 60
rainfall_mm	float	no	≥ 0 , ≤ 2000
sunshine_hours	float	no	≥ 0 , ≤ 750
snow_depth_cm	float	no	≥ 0
data_quality	integer	no	1, 2, or 3 (default: 1)

Example request:

```
POST /observations
X-API-Key: change-me-in-production
Content-Type: application/json

{
  "station_id": 5,
  "date": "2026-01-15",
  "max_temp_c": 10.2,
  "min_temp_c": 3.8,
  "mean_temp_c": 7.0,
  "rainfall_mm": 61.4,
  "sunshine_hours": 48.1
}
```

Error responses: 401, 404 (station not found), 409 (duplicate date+station), 422.

`PUT /observations/

Full replacement. All fields from POST body apply.

Error responses: 401, 404, 422.

`PATCH /observations/

Supply only changed fields.

Example request:

```
PATCH /observations/8834
X-API-Key: change-me-in-production
Content-Type: application/json

{
  "data_quality": 2,
  "rainfall_mm": 57.1
}
```

Error responses: 401, 404, 422.

`DELETE /observations/

Response: 204 No Content.

Error responses: 401, 404.

Annotations

Annotations are user-submitted notes attached to a station and a specific observation date. Submission is public; approval/editing requires an API key.

GET /annotations — List annotations

Query parameters:

Parameter	Type	Description
station_id	integer	Filter by station
approved	boolean	Filter by approval status
date_from	date	Filter from this observation date
date_to	date	Filter to this observation date
page / page_size	integer	Pagination

Example request:

```
GET /annotations?station_id=5&approved=true
```

Example response:

```
{
  "data": [
    {
      "id": 3,
      "station_id": 5,
      "observation_date": "2003-08-15",
      "author": "Dr. Smith",
      "note": "Record August heatwave. Maximum temperature reached 37.9°C at nearby Faversham.",
      "approved": true,
      "created_at": "2025-03-01T10:00:00",
      "updated_at": "2025-03-01T10:00:00"
    }
  ],
  "pagination": { "page": 1, "page_size": 50, "total": 1, "pages": 1 }
}
```

`GET /annotations/`

Error responses: 404.

POST /annotations — Submit an annotation (*public — no API key required*)

New annotations are unapproved by default.

Request body:

Field	Type	Required	Constraints
station_id	integer	yes	Must reference an existing station
observation_date	date	yes	YYYY-MM-DD
author	string	yes	Max 100 chars
note	string	yes	Max 2000 chars

Example request:

```
POST /annotations
Content-Type: application/json
```

```
{
  "station_id": 5,
  "observation_date": "2003-08-15",
  "author": "Dr. Smith",
  "note": "Record August heatwave. Hottest UK temperature on record."
}
```

Response: 201 Created — returns the annotation with "approved": false.

Error responses: 404 (station not found), 422.

PUT /annotations/

Error responses: 401, 404, 422.

PATCH /annotations/

Used to approve or reject annotations.

Example request:

```
PATCH /annotations/3
X-API-Key: change-me-in-production
Content-Type: application/json

{
  "approved": true
}
```

Error responses: 401, 404, 422.

DELETE /annotations/

Response: 204 No Content. **Error responses:** 401, 404.

Analytics

All analytics endpoints are **read-only** (no API key required). Rate limiting applies: 60 requests/minute per IP on computationally expensive endpoints.

Valid climate variables (variable parameter):

Value	Description
max_temp_c	Monthly mean daily maximum temperature (°C)
min_temp_c	Monthly mean daily minimum temperature (°C)
mean_temp_c	Monthly mean temperature (°C)
rainfall_mm	Total monthly rainfall (mm)
sunshine_hours	Total monthly sunshine hours

GET /analytics/trends/

Computes a linear regression over time for any climate variable at a station. Returns the slope (change per decade), R² goodness-of-fit, and all data points.
Uses PostgreSQL's native `regr_slope` and `regr_r2` aggregate functions.

Path parameters:

Parameter	Type	Description
station_id	string	Met Office station code, e.g. HEATHROW

Query parameters:

Parameter	Type	Required	Default	Description
variable	string	no	mean_temp_c	Climate variable
date_from	date	yes	—	Start date (YYYY-MM-DD)
date_to	date	yes	—	End date (YYYY-MM-DD)

Example request:

```
GET /analytics/trends/DURHAM?variable=mean_temp_c&date_from=1900-01-01&date_to=2025-12-31
```

Example response:

```
{
  "station_id": "DURHAM",
  "variable": "mean_temp_c",
  "date_from": "1900-01-01",
  "date_to": "2025-12-31",
  "slope_per_decade": 0.142,
  "r_squared": 0.312,
  "intercept": 6.84,
  "data_points": [
    { "date": "1900-01-15", "value": 2.1 },
    { "date": "1900-02-15", "value": 3.4 }
  ]
}
```

Interpretation: A `slope_per_decade` of 0.142 means the variable increases by 0.142°C every 10 years. `r_squared` of 0.312 indicates moderate linear correlation.

Error responses: 404 (station not found), 422 (invalid variable, `date_to` before `date_from`).

`GET /analytics/anomalies/

Identifies months where a climate variable deviates more than `threshold_sigma` standard deviations from the long-term monthly mean. The algorithm computes a z-score for each observation against the historical baseline for that calendar month.

Path parameters:

Parameter	Type	Description
station_id	string	Met Office station code

Query parameters:

Parameter	Type	Required	Default	Description
variable	string	no	max_temp_c	Climate variable
threshold_sigma	float	no	2.0	Detection threshold (0.5–5.0 σ)
date_from	date	no	—	Optional start date filter
date_to	date	no	—	Optional end date filter

Example request:

```
GET /analytics/anomalies/HEATHROW?variable=max_temp_c&threshold_sigma=3.0
```

Example response:

```
{
  "station_id": "HEATHROW",
  "variable": "max_temp_c",
  "threshold_sigma": 3.0,
  "anomalies": [
    {
      "id": 4521,
      "date": "1963-01-15",
      "value": 0.8,
      "monthly_mean": 8.3,
      "deviation_sigma": -3.84,
      "variable": "max_temp_c"
    },
    {
      "id": 6201,
      "date": "2003-08-15",
      "value": 28.1,
      "monthly_mean": 23.4,
      "deviation_sigma": 3.12,
      "variable": "max_temp_c"
    }
  ]
}
```

Interpretation: A deviation_sigma of -3.84 means January 1963 was 3.84 standard deviations colder than the average January — an extreme cold event.

Error responses: 404, 422.

GET /analytics/seasonal/

Returns month-by-month statistics (mean, min, max, standard deviation, record count) for a variable over a chosen year range. Includes season labels.

Query parameters:

Parameter	Type	Required	Default	Description
variable	string	no	mean_temp_c	Climate variable
year_from	integer	no	1990	Start year (1800–2100)
year_to	integer	no	2024	End year (1800–2100)

Example request:

```
GET /analytics/seasonal/OXFORD?variable=rainfall_mm&year_from=1991&year_to=2020
```

Example response:

```
{
  "station_id": "OXFORD",
  "variable": "rainfall_mm",
  "year_from": 1991,
  "year_to": 2020,
  "monthly_stats": [
    {
      "month": 1,
      "month_name": "January",
      "season": "Winter",
      "mean": 52.4,
      "min": 9.1,
      "max": 131.6,
      "std_dev": 28.2,
      "count": 30
    },
    {
      "month": 7,
      "month_name": "July",
      "season": "Summer",
      "mean": 44.1,
      "min": 5.2,
      "max": 107.3,
      "std_dev": 24.8,
      "count": 30
    }
  ]
}
```

Error responses: 404, 422.

GET /analytics/compare — Compare multiple stations

Returns aggregate statistics across 2 or more stations for a given variable and date range. Useful for understanding geographic climate variation.

Query parameters:

Parameter	Type	Required	Description
stations	string	yes	Comma-separated station IDs (min 2), e.g. HEATHROW, DURHAM, LERWICK
variable	string	no	Climate variable (default <code>mean_temp_c</code>)
date_from	date	yes	Start date
date_to	date	yes	End date

Example request:

```
GET /analytics/compare?stations=HEATHROW,DURHAM,LERWICK&variable=mean_temp_c&date_from=1991-01-01&date_to=2020-12-31
```

Example response:

```
{
  "variable": "mean_temp_c",
  "date_from": "1991-01-01",
  "date_to": "2020-12-31",
  "stations": [
    {
      "station_id": "HEATHROW",
      "station_name": "Heathrow",
      "mean": 11.8,
      "min": 1.2,
      "max": 24.1,
      "count": 360
    },
    {
      "station_id": "DURHAM",
      "station_name": "Durham",
      "mean": 9.1,
      "min": -1.4,
      "max": 20.6,
      "count": 360
    },
    {
      "station_id": "LERWICK",
      "station_name": "Lerwick",
      "mean": 7.4,
      "min": -0.8,
      "max": 16.2,
      "count": 360
    }
  ]
}
```

Error responses: 404 (any station not found), 422 (fewer than 2 stations, invalid dates).

GET /analytics/extremes — All-time climate records

Returns the single most extreme observation for each variable across all stations (or within a region).

Query parameters:

Parameter Type	Description
region	string Optional region filter (partial match)

Example request:

```
GET /analytics/extremes
```

Example response:

```
{
  "hottest_day": {
    "station_id": "CAMBRIDGE",
    "station_name": "Cambridge",
    "date": "2006-07-15",
    "value": 28.3
  },
  "coldest_day": {
    "station_id": "DURHAM",
    "station_name": "Durham",
    "date": "1963-01-15",
    "value": -7.4
  },
  "wettest_day": {
    "station_id": "CARDIFF",
    "station_name": "Cardiff",
    "date": "2000-11-15",
    "value": 237.8
  },
  "most_sunshine": {
    "station_id": "OXFORD",
    "station_name": "Oxford",
    "date": "1976-07-15",
    "value": 277.6
  }
}
```

`GET /analytics/heatmap/

Returns all monthly values for a variable and year, structured for rendering as a calendar heatmap. Each cell is one month with its observed value.

Query parameters:

Parameter	Type	Required	Description
variable	string	no	Climate variable (default <code>mean_temp_c</code>)
year	integer	yes	Year to retrieve (e.g. 2023)

Example request:

```
GET /analytics/heatmap/HEATHROW?variable=max_temp_c&year=2022
```

Example response:

```
{
  "station_id": "HEATHROW",
  "variable": "max_temp_c",
  "year": 2022,
  "cells": [
    { "date": "2022-01-15", "value": 9.1 },
    { "date": "2022-02-15", "value": 10.3 },
    { "date": "2022-07-15", "value": 32.1 },
    { "date": "2022-12-15", "value": 8.7 }
  ]
}
```

Error responses: 404, 422.

GET /analytics/climate-normal/

Returns the World Meteorological Organisation (WMO) standard 30-year climate normals for the period **1991–2020**. These are the internationally recognised baseline averages used to compare current conditions against historical norms.

Example request:

```
GET /analytics/climate-normal/HEATHROW
```

Example response:

```
{
  "station_id": "HEATHROW",
  "station_name": "Heathrow",
  "normal_period": "1991-2020",
  "normals": [
    {
      "month": 1,
      "month_name": "January",
      "avg_max_temp_c": 8.5,
      "avg_min_temp_c": 3.2,
      "avg_rainfall_mm": 54.1,
      "avg_sunshine_hours": 61.3
    },
    {
      "month": 7,
      "month_name": "July",
      "avg_max_temp_c": 23.6,
      "avg_min_temp_c": 14.1,
      "avg_rainfall_mm": 37.2,
      "avg_sunshine_hours": 205.4
    }
  ]
}
```

Error responses: 404.

System

GET /health — Health check

Returns the API and database status. Used for monitoring and container health checks.

Example request:

```
GET /health
```

Example response:

```
{
  "status": "ok",
  "database": "connected",
  "version": "1.0.0"
}
```

MCP Server

This API ships with a Model Context Protocol (MCP) (<https://modelcontextprotocol.io>) server (`mcp_server.py`), allowing AI assistants such as Claude Desktop to query the climate database directly as callable tools.

Available tools:

Tool	Maps to
<code>list_stations</code>	<code>GET /stations</code>
<code>get_climate_normals</code>	<code>GET /analytics/climate-normal/{id}</code>
<code>get_seasonal_stats</code>	<code>GET /analytics/seasonal/{id}</code>
<code>get_climate_trend</code>	<code>GET /analytics/trends/{id}</code>
<code>detect_anomalies</code>	<code>GET /analytics/anomalies/{id}</code>
<code>compare_stations</code>	<code>GET /analytics/compare</code>
<code>get_all_time_records</code>	<code>GET /analytics/extremes</code>
<code>get_observations</code>	<code>GET /observations</code>

See the `README.md` for Claude Desktop configuration instructions.